

Resolução do Quebra-Cabeça Tango por Força Bruta e Backtracking

Guilherme Brites, Murilo Andrade e Pedro Henrique

Curso de Engenharia de Software

Pontifícia Universidade Católica de Minas Gerais

Fundamentos de Projeto e Análise de Algoritmos, 2026/1

Resumo—Este trabalho modela o quebra-cabeça lógico Tango como um Problema de Satisfação de Restrições e o resolve, em Java, por duas estratégias: busca exaustiva (força bruta) e *backtracking* com poda. Ambas compartilham um único validador de regras, diferindo apenas no momento em que as restrições são avaliadas. Mostra-se que, embora as duas estratégias possuam o mesmo pior caso assintótico exponencial, a poda reduz o número de estados explorados em cerca de 940 vezes sobre um mesmo tabuleiro e viabiliza instâncias que a busca exaustiva jamais concluiria. Medições diretas indicam que as regras do jogo tornam válida apenas uma fração da ordem de uma em seis milhões das grades 6×6 possíveis, o que explica a eficácia da poda.

Index Terms—Backtracking, força bruta, poda, problemas de satisfação de restrições, análise de complexidade, Tango.

I. INTRODUÇÃO

Os Problemas de Satisfação de Restrições (CSPs) formam uma classe fundamental de desafios em que se busca um estado que satisfaça um conjunto de regras pré-definidas. Jogos lógicos e quebra-cabeças são laboratórios ideais para o estudo de CSPs, pois exigem a formulação rigorosa de estados e a aplicação de algoritmos de busca no espaço de soluções. A resolução ingênua desses problemas (a força bruta) testa todas as combinações possíveis, frequentemente incorrendo em explosão combinatória. O *backtracking* surge como refinamento, construindo soluções de forma incremental e aplicando podas assim que uma restrição é violada.

Este trabalho modela e resolve o quebra-cabeça Tango por ambas as técnicas. O Tango é jogado em uma grade $N \times N$ que deve ser preenchida com dois símbolos (Sol e Lua) respeitando cinco regras: (1) cada célula contém obrigatoriamente um símbolo; (2) não mais que dois símbolos idênticos podem ficar lado a lado, na horizontal ou na vertical; (3) cada linha e cada coluna contém exatamente a mesma quantidade de cada símbolo; (4) células ligadas por “=” contêm o mesmo símbolo; e (5) células ligadas por “×” contêm símbolos opostos. As seções seguintes detalham a modelagem, as duas estratégias de resolução, exemplos de execução e a análise de complexidade.

II. MODELAGEM DO PROBLEMA

O problema foi modelado separando três responsabilidades: o estado do tabuleiro, as restrições de aresta entre células e a validação das regras. Essa separação é o que permite que as duas estratégias de resolução compartilhem exatamente a mesma lógica de regras.

A. Representação do tabuleiro

O tabuleiro é representado pela classe `Board`, que encapsula uma matriz `int[][]` de dimensão $N \times N$. Cada célula assume um de três valores inteiros, definidos como constantes: `EMPTY = 0`, `SUN = 1` e `MOON = 2`.

A classe expõe operações de leitura e escrita célula a célula (`get`, `set`), consulta de dimensão (`getN`), verificação de vacância (`isEmpty`) e de completude (`isComplete`), além de uma cópia profunda (`copy`). A construção valida dimensão e domínio dos valores, garantindo que nenhum estado inconsistente seja representável.

B. Representação das restrições de igualdade e oposição

As restrições “=” e “×” não pertencem a uma célula, e sim à aresta entre duas células adjacentes. Por isso são modeladas à parte, na classe `EdgeConstraints`, por meio de duas matrizes:

- `h`, de dimensão $N \times (N - 1)$, onde `h[i][j]` descreve a aresta entre as células (i, j) e $(i, j+1)$: as restrições horizontais.
- `v`, de dimensão $(N - 1) \times N$, onde `v[i][j]` descreve a aresta entre (i, j) e $(i+1, j)$: as restrições verticais.

As dimensões assimétricas refletem o fato geométrico de que uma linha de N células possui apenas $N - 1$ espaços internos entre elas. Cada posição assume `NONE = 0`, `EQUAL = 1` ou `OPPOSITE = 2`. Essa modelagem matricial foi escolhida em vez de uma estrutura associativa (por exemplo, um `Map` de pares de coordenadas) porque o acesso a uma restrição é feito por indexação direta, em tempo $O(1)$, sem cálculo de *hash*; propriedade relevante já que a poda consulta arestas a cada tentativa de preenchimento.

C. Encapsulamento da entrada

A leitura é responsabilidade de `BoardReader`, que interpreta o arquivo de texto (dimensão N , grade inicial com dicas, e os blocos `H`: e `V`: de restrições), tolera linhas em branco, valida que N é par (condição necessária para a regra de equilíbrio) e lança exceção com mensagem clara diante de qualquer desvio de formato. O resultado é entregue como um `record Puzzle(Board board, EdgeConstraints constraints)`, agregando estado e restrições em um único objeto imutável. A impressão em terminal, com as lacunas e os marcadores de restrição posicionados entre as células, fica a cargo de `BoardPrinter`.

D. Organização modular

O projeto separa a lógica de validação das regras (pacote `validation`, classe `RuleValidator`) da mecânica de busca (pacote `solver`). O validador é *stateless* e, portanto, uma única instância é compartilhada pelos dois *solvers*. Essa decisão atende diretamente ao requisito de modularização do enunciado e materializa a ideia central do trabalho: força bruta e *backtracking* diferem apenas em *quando* invocam o validador, nunca em *quais* regras aplicam.

III. ESTRATÉGIA DE RESOLUÇÃO: FORÇA BRUTA

A força bruta resolve o problema por busca exaustiva: enumera todo o espaço de preenchimentos possíveis e, para cada um, verifica se constitui uma solução válida. É a abordagem ingênua que serve de referência para evidenciar, por contraste, o ganho obtido pela poda.

A. Geração do espaço de estados

Antes de iniciar a busca, o `BruteForceSolver` percorre o tabuleiro e coleta a lista das k células vazias, as posições que ainda precisam ser preenchidas, já que as dicas iniciais são fixas. Como cada uma dessas células só pode receber Sol ou Lua, o número total de tabuleiros possíveis é 2^k : dois valores elevado à quantidade de células livres.

Para gerar todas essas combinações sem esquecer nenhuma nem repetir, o *solver* usa um contador que vai de 0 até $2^k - 1$ e trata cada número como uma sequência de bits. Cada bit responde por uma célula vazia: se o bit vale 0, aquela célula recebe um símbolo; se vale 1, recebe o outro. Assim, percorrer todos os números de 0 a $2^k - 1$ é o mesmo que percorrer todas as formas possíveis de preencher o tabuleiro: cada número gera exatamente um preenchimento diferente.

B. Momento da validação

A característica que define a força bruta é que a validação só ocorre sobre o tabuleiro **completo**. A cada combinação gerada, todas as k células são preenchidas e só então o método `isFullyValid` do `RuleValidator` é invocado, avaliando as cinco regras de uma vez, inclusive a Regra 1 (preenchimento completo), que aqui é trivialmente satisfeita por construção. O algoritmo não faz nenhum julgamento parcial durante a montagem da combinação: monta o estado inteiro às cegas e pergunta, ao final, se ele é válido. Adota-se *early-exit*: a primeira combinação aprovada é retornada como solução, e como um tabuleiro de Tango bem formulado tem solução única, essa é necessariamente a resposta correta.

C. Guarda de inviabilidade

Como 2^k cresce exponencialmente, a busca torna-se impraticável já para tabuleiros com poucas dezenas de células vazias. Para evitar que o programa entre em execução interminável, o *solver* aplica uma guarda de segurança: se k excede um limite (`MAX_EMPTY = 24`), a enumeração não é iniciada; o *solver* sinaliza o aborto e retorna sem travar o restante do programa. Essa salvaguarda é o reconhecimento explícito, no próprio código, da explosão combinatória que a próxima estratégia se propõe a contornar.

IV. ESTRATÉGIA DE RESOLUÇÃO: BACKTRACKING

O *backtracking* refina a busca exaustiva construindo a solução de forma incremental e descartando ramos inteiros do espaço de estados assim que detecta que nenhuma de suas folhas poderá ser válida. A diferença em relação à força bruta não está nas regras avaliadas, e sim no **momento** da avaliação: aqui as regras são consultadas a cada célula preenchida, antes de prosseguir.

A. A função recursiva

A busca é uma função recursiva indexada pelas células em ordem *row-major*, de 0 a $N \cdot N - 1$. Em cada chamada:

- **Condição de parada (base):** quando o índice alcança $N \cdot N$, todas as células foram preenchidas respeitando as regras a cada passo, e o tabuleiro está, por construção, resolvido. A recursão reporta sucesso.
- **Célula com dica:** se a posição atual já vem preenchida pela entrada, ela é fixa; a função apenas avança para o índice seguinte, sem tentar valores.
- **Célula vazia:** o algoritmo tenta os dois símbolos (Sol e depois Lua). Para cada tentativa, atribui o valor e chama o validador parcial; se a tentativa for aceita, recursa para a próxima célula; se a recursão subsequente falhar, ou se a própria tentativa for rejeitada de imediato, o valor é desfeito (*backtrack*) e o outro símbolo é tentado. Esgotados os dois sem sucesso, a chamada retorna falha, fazendo o nível anterior retroceder.

A atribuição é feita por mutação no próprio tabuleiro, com desfazimento no retrocesso, evitando a cópia do estado a cada nó, o que preservaria correção mas degradaria o desempenho que a poda busca conquistar.

B. As cinco regras como critérios de poda

O coração da estratégia é o `isPartialValid`: diferentemente do `isFullyValid` da força bruta, ele não exige tabuleiro completo (a Regra 1 não se aplica durante a construção) e avalia **apenas o que a jogada recém-feita em (linha, coluna) pode ter violado**, sem varrer o tabuleiro inteiro. Cada regra contribui com um critério de poda:

- **Regra 2 (adjacência):** verifica somente as janelas de três células que contêm a posição recém-preenchida, na horizontal e na vertical, rejeitando a formação de três símbolos iguais consecutivos. Como o preenchimento é *row-major*, as células à direita e abaixo ainda estão vazias no momento da jogada, então basta inspecionar as janelas que terminam na célula atual.
- **Regra 3 (equilíbrio):** conta os símbolos já presentes na linha e na coluna da jogada. Poda imediatamente se algum símbolo ultrapassa $N/2$ ocorrências; e aplica um *lookahead*: se as células ainda vazias da linha ou da coluna forem insuficientes para acomodar o que falta do outro símbolo, o ramo é abandonado. Esse antecipar de impasses é o que impede o algoritmo de descobrir becos sem saída apenas no fundo da recursão.

- **Regras 4 e 5 (igualdade e oposição):** inspecionam as arestas que incidem sobre a célula atual, comparando-a com vizinhos **já preenchidos** e ignorando os ainda vazios. Uma restrição “=” exige símbolos iguais; uma restrição “x”, símbolos opostos. A violação de qualquer aresta determinada poda o ramo.

A cada nó, portanto, o algoritmo só desce na árvore de busca se o estado parcial permanece consistente com todas as restrições aplicáveis até ali. Ramos que a força bruta exploraria por completo são eliminados na raiz, e é essa eliminação antecipada, e não uma mudança nas regras, que produz a diferença de ordens de grandeza no esforço de busca, quantificada na Seção VI.

V. EXEMPLOS DE EXECUÇÃO

Esta seção apresenta o programa operando sobre tabuleiros distintos e com ambas as estratégias, cobrindo os principais cenários de uso: resolução por *backtracking*, resolução por força bruta, o disparo da guarda de inviabilidade e o tratamento de um tabuleiro de tamanho diferente sem solução.

A. Backtracking resolvendo um tabuleiro 6×6

A execução abaixo, invocada com `tango.Main input/exemplo6_solvavel.txt bt`, resolve o tabuleiro de 32 células vazias. O programa imprime a configuração inicial (com as lacunas e os marcadores de restrição entre as células), executa o *backtracking* e imprime o tabuleiro final, seguido das métricas.

```
Tabuleiro inicial (6x6):
L   .   . x .   .   .
=
. = .   .   .   .   .
      x
.   .   . S   . = .
x
. L x .   . S   .
      x
.   .   . x .   .
=
.   .   . x .   .

Legenda: S=Sol L=Lua .=vazio ==iguais x=opostas

=== Backtracking ===
Tabuleiro final (resolvido):

L   L   S x L   S   S
=
L = L   S   L   S   S
      x
S   S   L   S   L = L
x
L   L x S   L   S   S
      x
S   S   L   S x L   L
=
S   S   L x S   L   L

Legenda: S=Sol L=Lua .=vazio ==iguais x=opostas
Nos explorados: 362
Tempo: 0 ms
```

A solução respeita as cinco regras e foi alcançada com 362 tentativas de atribuição, ínfimo perante as $2^{32} \approx 4,3$ bilhões de configurações do espaço completo.

B. Força bruta resolvendo um tabuleiro 6×6

Para mostrar a busca exaustiva resolvendo de fato, a execução abaixo (`tango.Main input/exemplo6_bruteforce.txt bf`) usa um tabuleiro de apenas 16 células vazias, dentro do limite viável. O programa enumera as combinações até encontrar a primeira válida.

```
Tabuleiro inicial (6x6):
.   .   S x .   S   .
=
L = L   S   .   .   S
      x
S   S   L   S   L = L
x
.   L x .   .   .   S
      x
S   S   .   S x .   .
=
.   .   . x S   L   .

Legenda: S=Sol L=Lua .=vazio ==iguais x=opostas

=== Força Bruta ===
Tabuleiro final (resolvido):

L   L   S x L   S   S
=
L = L   S   L   S   S
      x
S   S   L   S   L = L
x
L   L x S   L   S   S
      x
S   S   L   S x L   L
=
S   S   L x S   L   L

Legenda: S=Sol L=Lua .=vazio ==iguais x=opostas
Celulas vazias (k): 16
Combinacoes testadas: 52824
Tempo: 18 ms
```

A solução obtida é idêntica à que o *backtracking* encontra para o mesmo conjunto de restrições, confirmando a correteza de ambas as estratégias. O contraste de esforço (52.824 combinações contra as 56 tentativas do *backtracking* sobre este mesmo tabuleiro) é analisado quantitativamente na Seção VI.

C. A guarda de inviabilidade da força bruta

Submetido ao tabuleiro de 32 células vazias com a estratégia de força bruta (`tango.Main input/exemplo6_solvavel.txt bf`), o programa reconhece que o número de células vazias excede o limite seguro e aborta apenas essa estratégia, sem travar.

```
Tabuleiro inicial (6x6):
[ ... mesma configuracao inicial da execucao anterior ... ]

=== Força Bruta ===
Estrategia inviavel: k=32 celulas vazias -> 2^32 =
4294967296 combinacoes.
Acima do limite de segurancia (k > 24); forca bruta abortada
(o programa segue normalmente).
```

Este recorte documenta o tratamento explícito da explosão combinatória: a busca exaustiva só é tentada quando se comporta em tempo praticável.

D. Tabuleiro de tamanho diferente (8×8)

Para evidenciar que a implementação é genérica para qualquer N par, a execução a seguir (tango.Main input/exemplo8.txt bt) carrega um tabuleiro 8×8 . Este tabuleiro foi usado como caso de teste de formato e não possui solução; a saída demonstra que o *backtracking* esgota corretamente o espaço de busca e reporta a ausência de solução, em vez de falhar.

```

Tabuleiro inicial (8x8):
S = . . L . . x .
      x
. . . x . . . .
      =
. . S . . = . L
      x
. . x . . . . .
=
. L . . . S = .
      x
. . . . = . . =
L x . . S . . .
      x
. . . . x . S

Legenda: S=Sol L=Lua .=vazio ==iguais x=opostas

=== Backtracking ===
Sem solucao: o espaco de busca foi esgotado sem tabuleiro
valido.
Nos explorados: 1540
Tempo: 0 ms

```

O alinhamento das restrições entre células permanece correto na grade maior, confirmando que o módulo de impressão e o *solver* não pressupõem o tamanho 6×6 .

VI. ANÁLISE DE COMPLEXIDADE

Esta seção discute o tamanho do espaço de busca gerado pelo problema, o custo assintótico de cada estratégia e, sobretudo, o efeito das regras do jogo na restrição desse espaço, comparando a previsão teórica com as medições obtidas pela própria implementação.

A. O espaço de busca

Para um tabuleiro $N \times N$ com k células vazias, cada célula admite dois símbolos, de modo que o espaço de preenchimentos possíveis tem tamanho 2^k . No pior caso, um tabuleiro sem nenhuma dica inicial, temos $k = N^2$ e o espaço chega a 2^{N^2} . Para um modesto tabuleiro 6×6 , isso significa $2^{36} \approx 6,87 \times 10^{10}$ configurações: a explosão combinatória que motiva o trabalho.

B. Custo da força bruta

A força bruta percorre integralmente esse espaço. Para cada uma das 2^k combinações, preenche as k células e executa *isFullyValid*, que avalia as cinco regras sobre o tabuleiro completo em tempo $O(N^2)$: varrendo células para a adjacência, somando linhas e colunas para o equilíbrio e percorrendo as arestas para “=” e “×”. O custo total é, portanto:

$$T_{\text{bruta}}(N, k) = O(2^k \cdot N^2) \quad (1)$$

No pior caso ($k = N^2$), $O(2^{N^2} \cdot N^2)$. O termo exponencial domina qualquer polinômio, e é por isso que a estratégia só é viável quando k é pequeno, situação garantida na prática pela guarda de segurança `MAX_EMPTY = 24`.

C. Custo do backtracking

O *backtracking* organiza a busca como uma árvore binária de profundidade k : cada nível corresponde a uma célula vazia e cada nó ramifica nas duas escolhas de símbolo. Em cada nó, a poda invoca *isPartialValid*, cujo custo é dominado pela Regra 3 (contagem da linha e da coluna da jogada), em tempo $O(N)$. No pior caso teórico, a árvore tem $O(2^k)$ nós e o custo é:

$$T_{\text{back}}(N, k) = O(2^k \cdot N) \quad (2)$$

É importante o rigor aqui: **assintoticamente, no pior caso, o backtracking não supera a força bruta**: ambos permanecem exponenciais em k . Um problema construído sem nenhuma oportunidade de poda forçaria a exploração da árvore inteira. O ganho do *backtracking* não está na classe assintótica de pior caso, e sim no comportamento prático sobre instâncias bem formuladas: a poda elimina ramos cedo, reduzindo o número de nós efetivamente visitados em ordens de grandeza. A diferença, portanto, é de fator constante e de caso esperado, mas, como mostram os números a seguir, é uma diferença enorme.

D. Como as regras restringem o espaço

A poda é tão eficaz porque as regras eliminam quase todos os tabuleiros possíveis: sobra só uma parcela minúscula que realmente respeita o jogo. Dá para medir isso diretamente no caso 6×6 .

Comece por uma única linha. Existem $2^6 = 64$ formas de preenchê-la com Sol e Lua. Mas a regra do equilíbrio (três de cada) deixa passar só 20 delas, e a regra da adjacência (nada de três iguais seguidos) corta mais algumas, sobrando apenas **14** linhas válidas. Ou seja, antes mesmo de pensar no tabuleiro inteiro, mais de três quartos das formas de montar uma linha já estão descartadas.

O efeito se multiplica quando juntamos as seis linhas e exigimos que as colunas também obedeçam às mesmas regras. Das $2^{36} \approx 68,7$ bilhões de maneiras de preencher uma grade 6×6 , só **11.222** são válidas sob equilíbrio e adjacência, cerca de **uma em cada seis milhões**, e isso **sem nem usar** as restrições “=” e “×”. As regras de aresta entram por cima desse punhado: cada “=” ou “×” amarra uma célula ao seu vizinho e, no *backtracking*, costuma forçar a única escolha possível ou eliminar uma das duas na hora. Somadas as cinco regras, sobra um único tabuleiro: a solução.

Resumindo o papel de cada regra como poda: o **equilíbrio** (Regra 3) é o mais forte, porque limita pela metade quantos símbolos de cada tipo cabem em cada linha e coluna, e ainda antecipa becos sem saída com o *lookahead*; a **adjacência** (Regra 2) barra sequências de três iguais; e as **regras de igualdade e oposição** (4 e 5) fixam a relação entre vizinhos, podendo reduzir uma jogada a uma só opção, ou a nenhuma.

Tabela I
COMPARAÇÃO EMPÍRICA ENTRE AS ESTRATÉGIAS SOBRE O MESMO
TABULEIRO. FORÇA BRUTA MEDIDA EM COMBINAÇÕES TESTADAS;
backtracking, EM NÓS EXPLORADOS.

Tabuleiro	k	Espaço 2^k	F. Bruta / Backtr.
bruteforce	16	65.536	52.824 / 56
solvavel	32	$\approx 4,29 \times 10^9$	inviável / 362

E. Verificação empírica

As medições da implementação confirmam a análise. Sobre o mesmo tabuleiro, instrumentamos o número de estados que cada estratégia processa: combinações testadas, na força bruta; tentativas de atribuição, no *backtracking*. A Tabela I resume os resultados.

No tabuleiro de 16 vazias, a força bruta processa **52.824** combinações completas, cada uma com validação $O(N^2)$, enquanto o *backtracking* chega à solução com **56** tentativas, cada uma com validação parcial $O(N)$. A redução é de aproximadamente **940**× no número de estados, e ainda maior no total de operações, já que o custo por estado também é menor no *backtracking*. No tabuleiro de 32 vazias, o contraste é categórico: a força bruta é inviável ($2^{32} \approx 4,29 \times 10^9$ combinações, barradas pela guarda de segurança), ao passo que o *backtracking* resolve o problema com apenas **362** tentativas: evidência de que a poda não apenas acelera, mas viabiliza instâncias que a busca exaustiva jamais concluiria.

F. Sensibilidade à ordem de tentativa

Uma observação final, de natureza prática: o número de nós explorados pelo *backtracking* depende da **ordem em que os símbolos são tentados** em cada célula. No tabuleiro de 32 vazias, tentar Sol antes de Lua resulta em 362 tentativas, ao passo que tentar Lua primeiro resulta em apenas 48, porque a solução desse tabuleiro é “Lua-pesada” nas primeiras células, e a ordem que acerta mais cedo retrocede menos. Essa variação afeta o fator constante, **nunca a corretude nem a classe assintótica**: ambas as ordens encontram a mesma solução única. Heurísticas mais sofisticadas de ordenação de variáveis e valores (como escolher primeiro a célula mais restrita) poderiam reduzir ainda mais o esforço, ao custo de abandonar o preenchimento *row-major* que mantém o validador parcial simples: um compromisso deliberado entre simplicidade e otimalidade da busca.